

Hadoop



Agenda

- Analogy
- Hadoop Introduction
- Hadoop History
- Hadoop Components
- Hadoop Architecture
- Hadoop Eco System

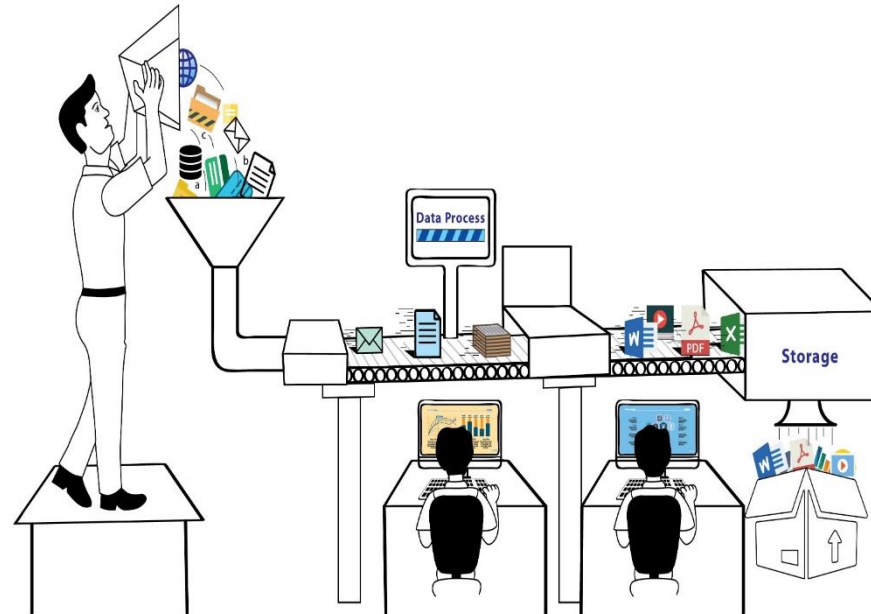
Recap (Problems of Big data)

1. how to store Huge Data?



Recap (Problems of Big data)

1. how to store Huge Data?
2. How to process Huge Data?



History of Hadoop

- Hadoop was started with **Doug Cutting and Mike Cafarella** in the year 2002

History of Hadoop

- Hadoop was started with **Doug Cutting and Mike Cafarella** in the year 2002
- **In 2003**, they came across a paper that described the architecture of Google's distributed file system, called **GFS (Google File System)** which was published by Google, for storing the large data sets

History of Hadoop

- Hadoop was started with **Doug Cutting and Mike Cafarella** in the year 2002
- **In 2003**, they came across a paper that described the architecture of Google's distributed file system, called **GFS (Google File System)** which was published by Google, for storing the large data sets
- **In 2004**, Google published one more paper on technique **MapReduce**, which was the solution of processing those large datasets

History of Hadoop

- Hadoop was started with **Doug Cutting and Mike Cafarella** in the year 2002
- **In 2003**, they came across a paper that described the architecture of Google's distributed file system, called **GFS (Google File System)** which was published by Google, for storing the large data sets
- **In 2004**, Google published one more paper on technique **MapReduce**, which was the solution of processing those large datasets
- These both techniques (**GFS & MapReduce**) were just on white paper at Google. Google didn't implement these two techniques

History of Hadoop

- So **in 2006**, Doug Cutting joined Yahoo

History of Hadoop

- So **in 2006**, Doug Cutting joined Yahoo
- He wanted to provide the world with an open-source, reliable, scalable computing framework, with the help of Yahoo

History of Hadoop

- So **in 2006**, Doug Cutting joined Yahoo
- He wanted to provide the world with an open-source, reliable, scalable computing framework, with the help of Yahoo
- So at Yahoo first, he separates the distributed computing parts and formed a new project **Hadoop**

History of Hadoop

- So **in 2006**, Doug Cutting joined Yahoo
- He wanted to provide the world with an open-source, reliable, scalable computing framework, with the help of Yahoo
- So at Yahoo first, he separates the distributed computing parts and formed a new project **Hadoop**
 - (He gave **name Hadoop** it was the name of a **yellow toy elephant which was owned by the Doug Cutting's son**. and it was easy to pronounce and was the unique word.)

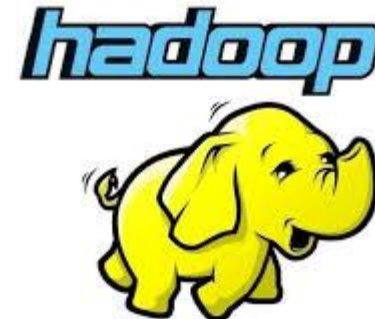
History of Hadoop

- So **in 2006**, Doug Cutting joined Yahoo
- He wanted to provide the world with an open-source, reliable, scalable computing framework, with the help of Yahoo
- So at Yahoo first, he separates the distributed computing parts and formed a new project **Hadoop**
 - (He gave **name Hadoop** it was the name of a **yellow toy elephant which was owned by the Doug Cutting's son**. and it was easy to pronounce and was the unique word.)



History of Hadoop

- So **in 2006**, Doug Cutting joined Yahoo
- He wanted to provide the world with an open-source, reliable, scalable computing framework, with the help of Yahoo
- So at Yahoo first, he separates the distributed computing parts and formed a new project **Hadoop**
 - (He gave **name Hadoop** it was the name of a **yellow toy elephant which was owned by the Doug Cutting's son**. and it was easy to pronounce and was the unique word.)



Hadoop

- Hadoop is an open source framework overseen by Apache Software Foundation which is written in Java for storing and processing of huge datasets with the cluster of commodity hardware
- There are mainly two components of Hadoop which are
 - **Hadoop Distributed File System (HDFS)** and
 - **Map Reduce (MR)**

HDFS

(Hadoop Distributed File System)

- HDFS is a distributed file system that handles large data sets running on commodity hardware
- HDFS holds very large amount of data and provides easier access
- When HDFS takes in data, it breaks the information down into separate blocks and distributes them to different nodes in a cluster, thus enabling highly *efficient parallel processing*

Features of HDFS

- Provides distributed storage
- Can be implemented on commodity hardware
- Provides data security
- Highly fault-tolerant - If one machine goes down, the data from that machine goes to the next machine

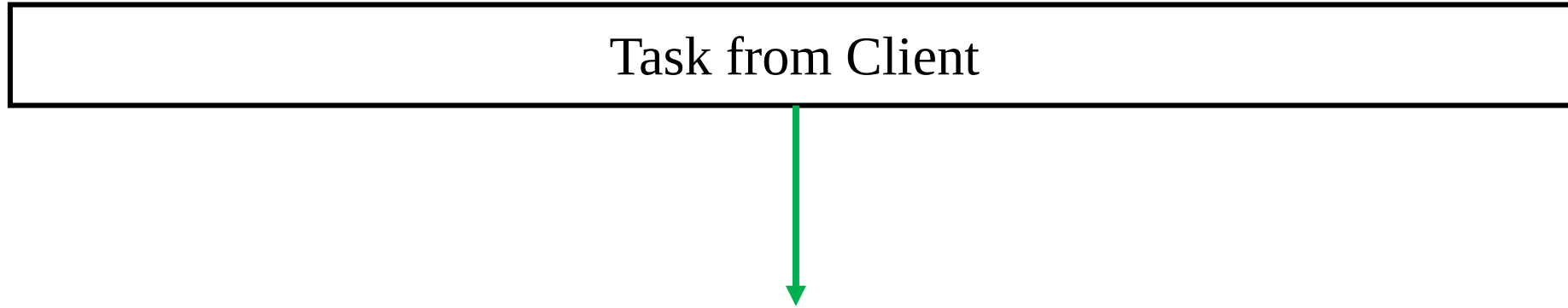
Map Reduce

- MapReduce is programming model used for processing huge amounts of data
- **MapReduce** program work in two phases, namely, Map and Reduce
- Map tasks deal with splitting and mapping of data while Reduce tasks shuffle and reduce the data
- With MapReduce, rather than sending data to where the application or logic resides, the logic is executed on the server where the data already resides, to expedite processing

Single Task Processing

Task from Client

Single Task Processing



Single Task Processing

Task from Client



Single Task Processing

Task from Client

Number of Tasks - 1



Single Task Processing

Task from Client



Number of Tasks - 1
Time Taken - 3 days



Single Task Processing

Task from Client



Number of Tasks - 1
Time Taken - 3 days
Number of Resource - 1

Single Task Processing

Task from Client



Number of Tasks - 1

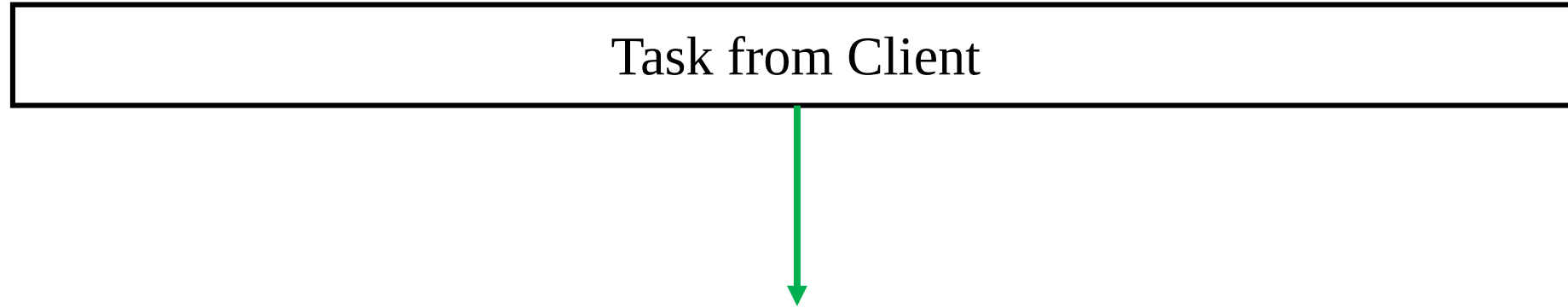
Time Taken - 3 days

Number of Resource - 1

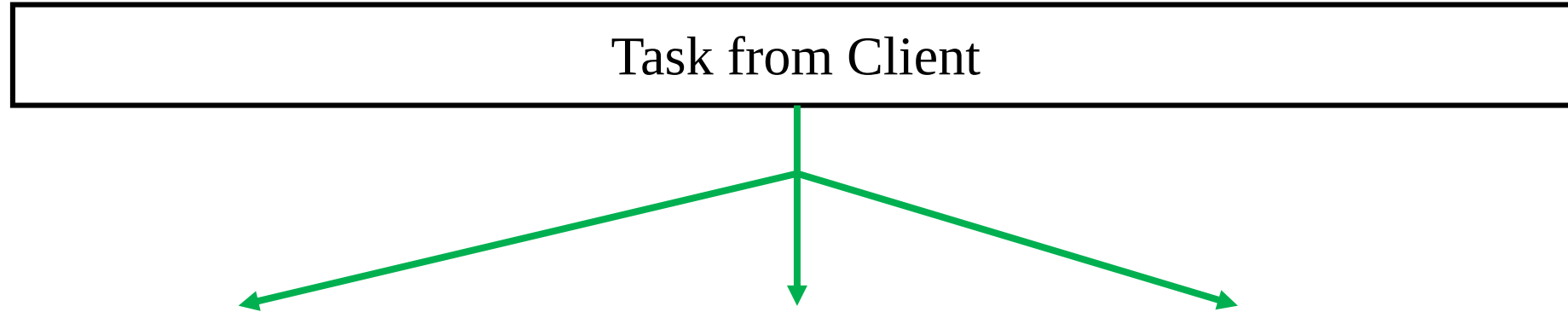
Multi -Task Processing

Task from Client

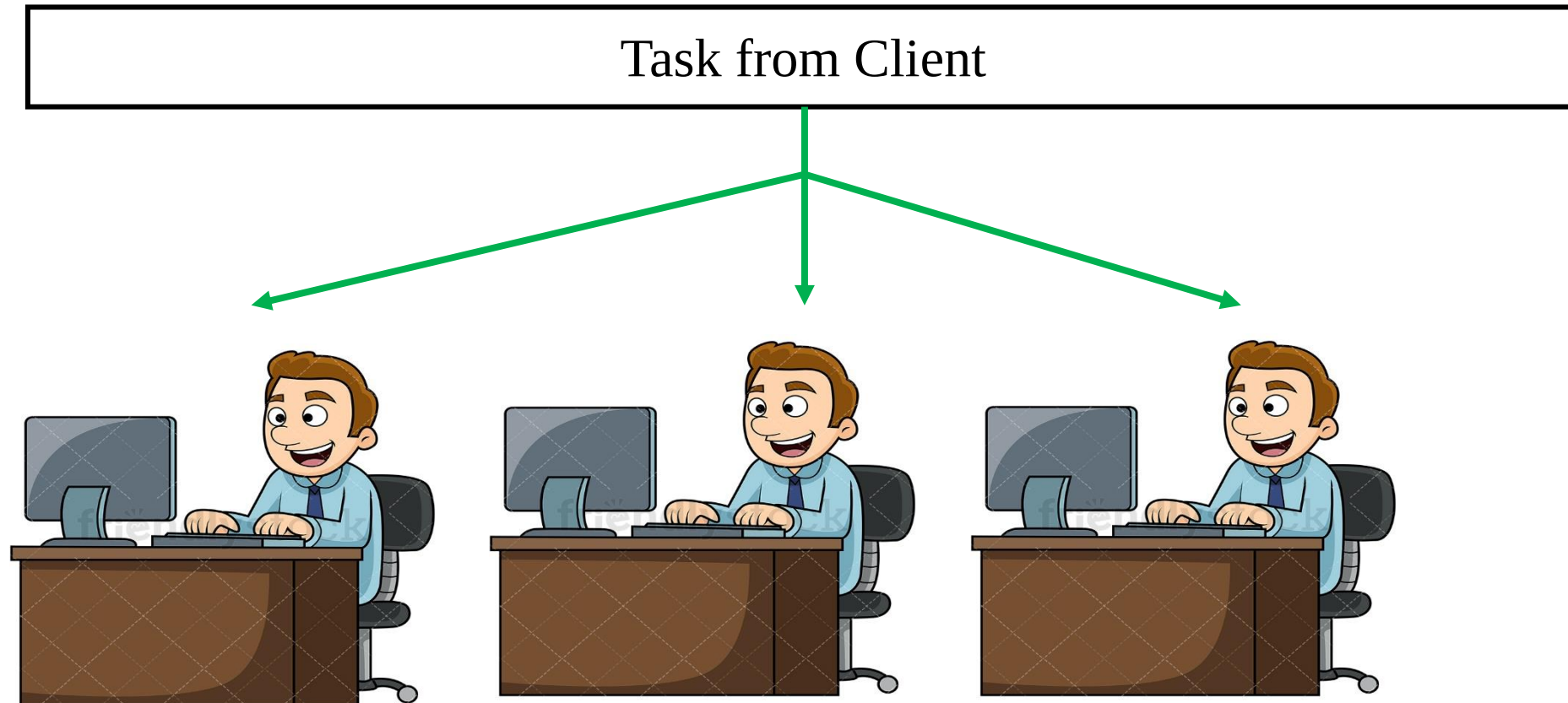
Multi -Task Processing



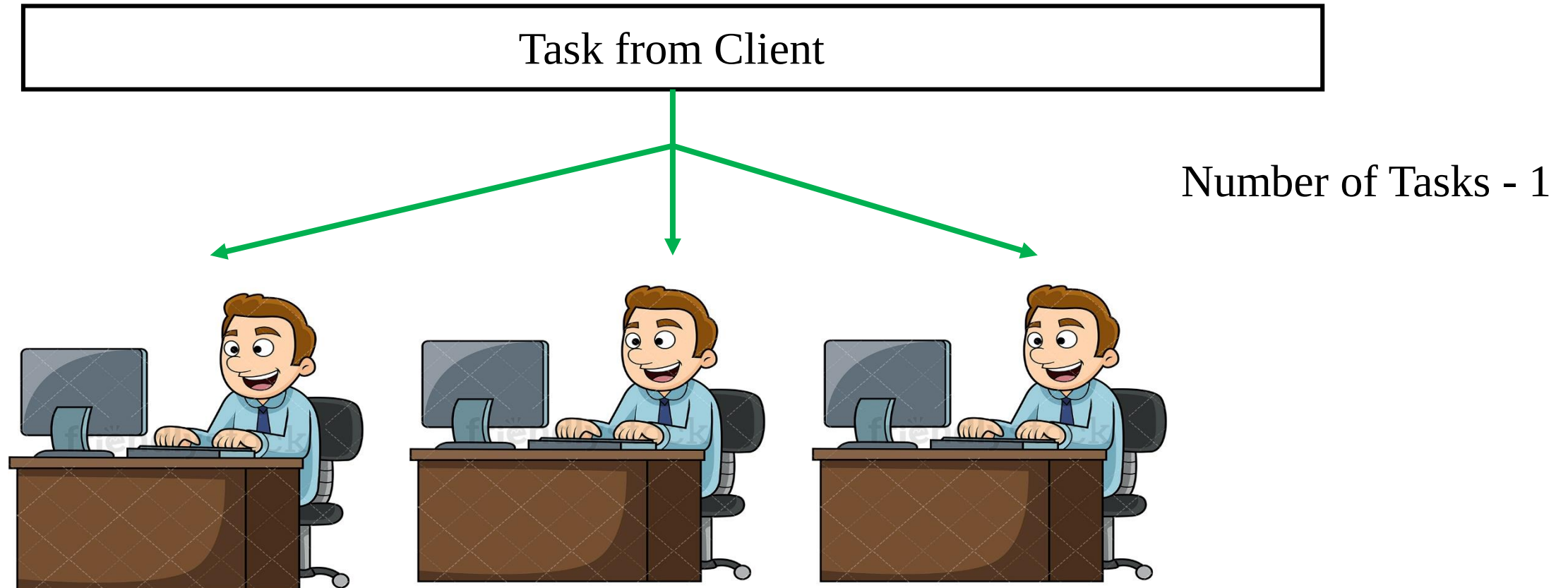
Multi -Task Processing



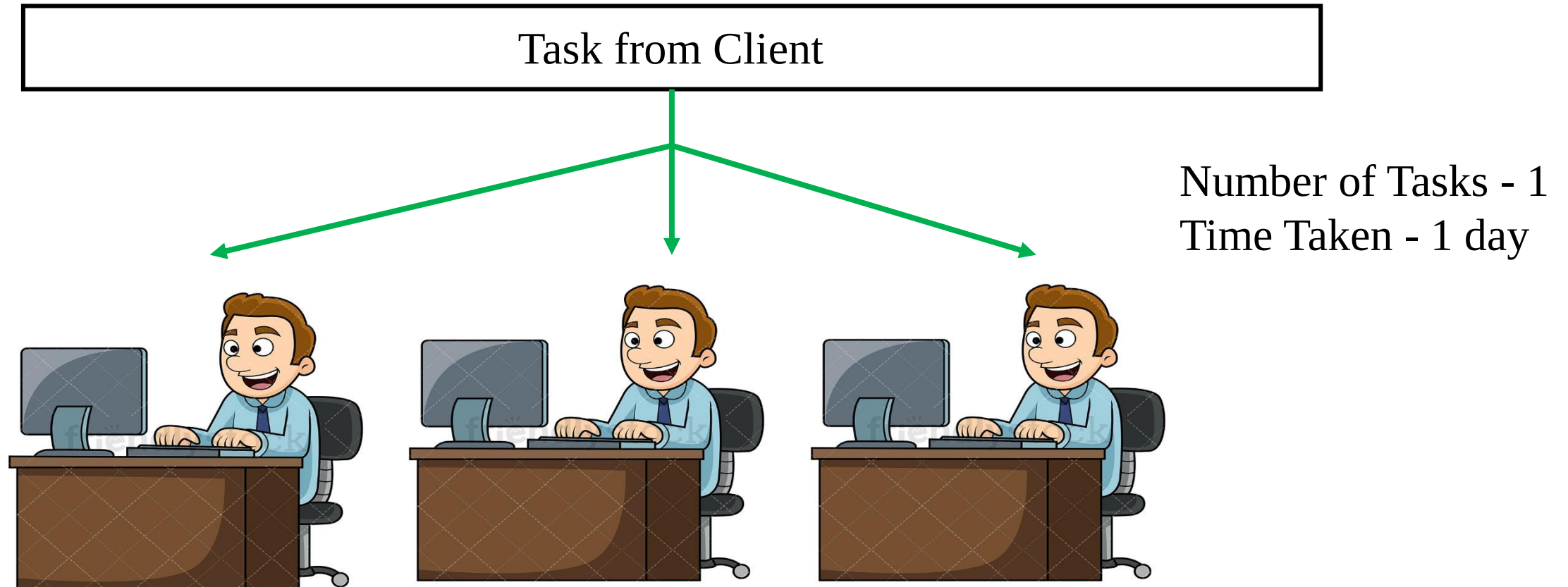
Multi -Task Processing



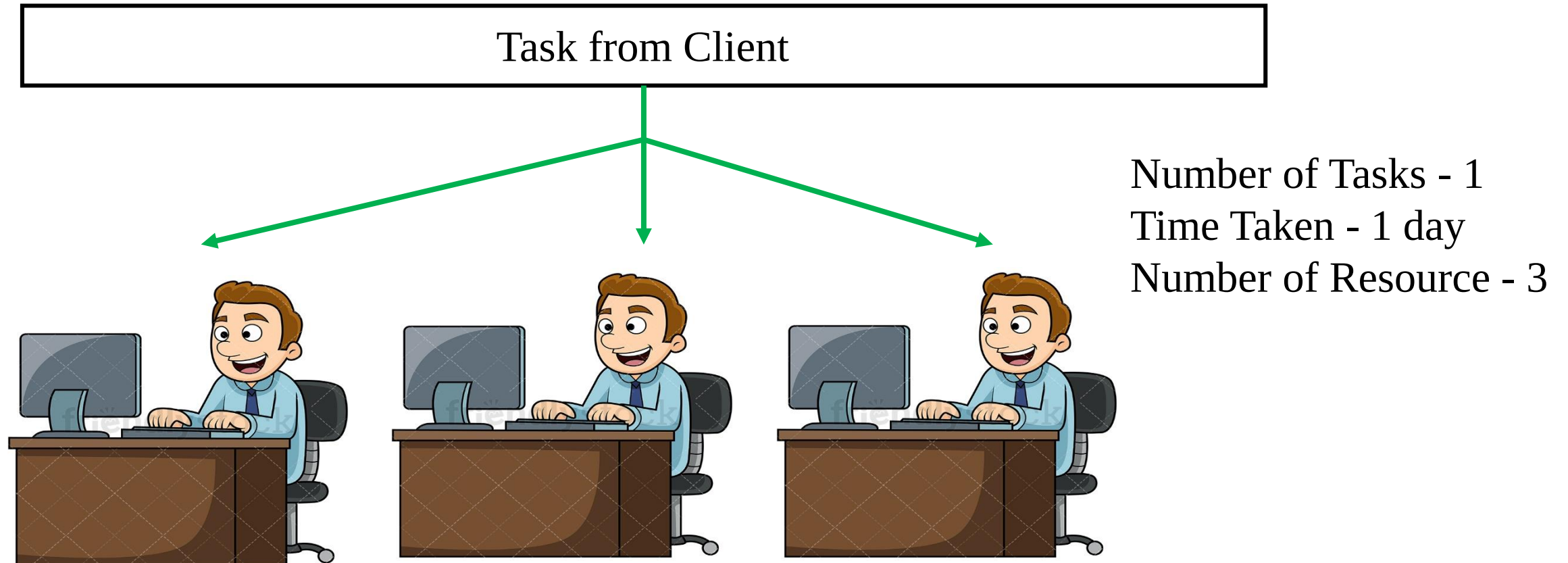
Multi -Task Processing



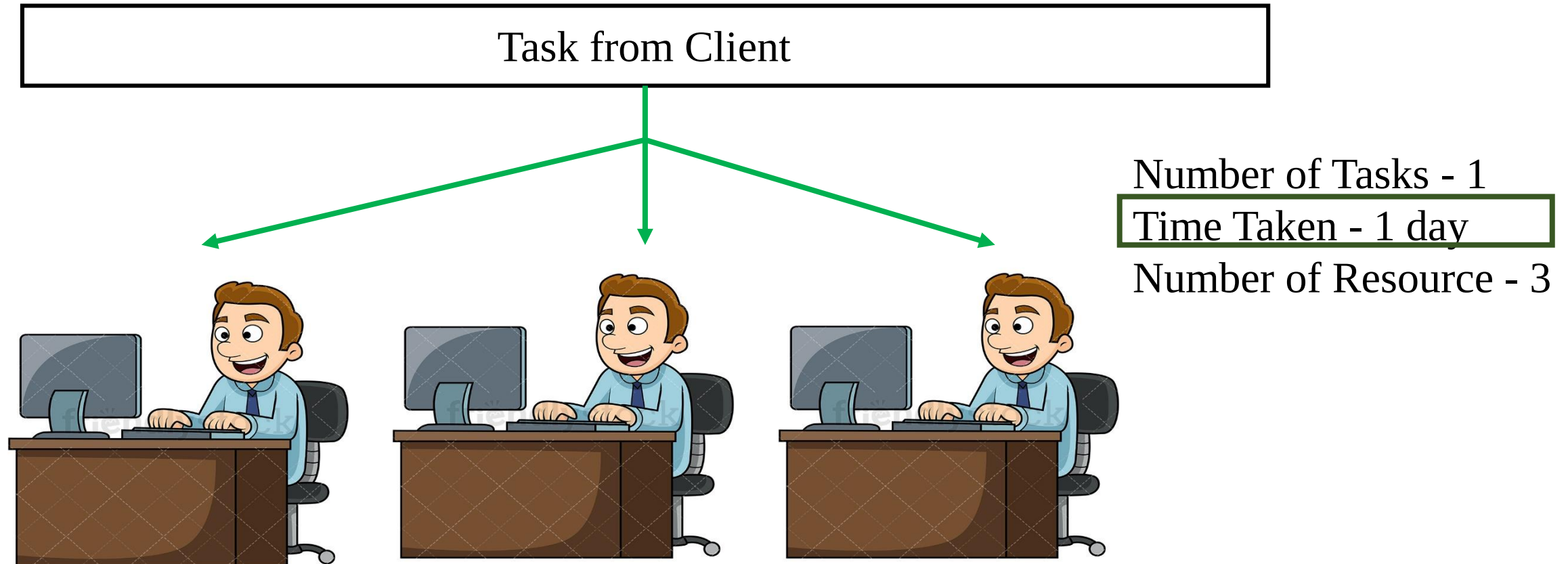
Multi -Task Processing



Multi -Task Processing



Multi -Task Processing



Hadoop Components

- Hadoop architecture can be categorized into three - Client Machines, Master Servers and Slave Servers. These are basically daemons or programs that run on different physical servers
- NameNode
- DataNode
- Secondary NameNode
- JobTracker
- TaskTracker

Name Node

- The Name Node is the centralized piece of the HDFS.
- It known as the *Master* and it is designed to store the *Meta Data*.
- Name Node is responsible for monitoring the *Health Status* of the *Slave Nodes* and to assign *Tasks* to the Data Nodes

Secondary Name Node

- As the name speaks, the Secondary Name Node is not a backup of the name node.
- It acts as a *Buffer* to the Name Node
- It stores the intermediate updates the *FS-image* of the Name Node in the *Edit-log* and updates the information to the Final *FS-image* when the name node is inactive

Data Node

- Data Node is the actual unit which stores the data.
- It is known as the *Slave* and it responds to the Name Node about its Health Status and the task status in the form of a *Heartbeat*.
- If the Data Node fails to respond to the Name Node, then the Name Node considers the Slave Node to be Dead and reassigns the task to the Next available Data Node

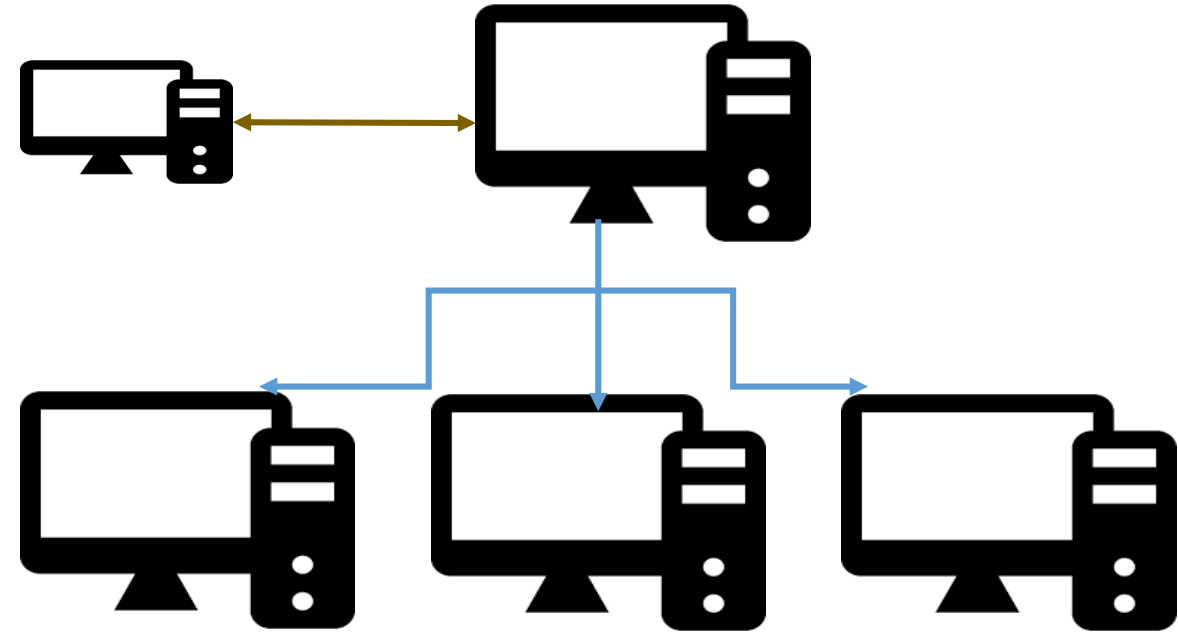
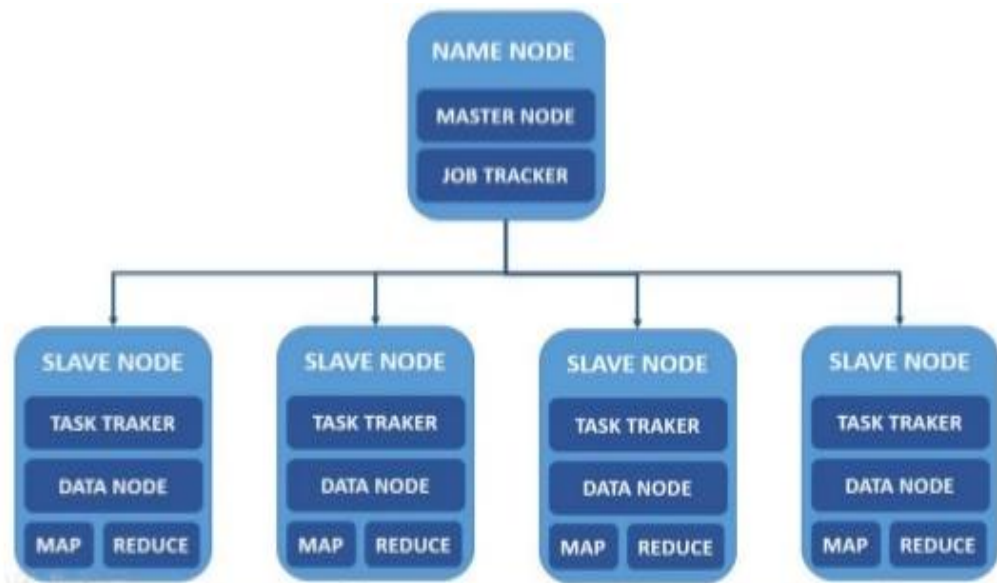
Job Tracker

- JobTracker process runs on a separate node and not usually on a DataNode
- JobTracker receives the requests for execution from the client.
- JobTracker talks to the NameNode to determine the location of the data.
- JobTracker finds the best TaskTracker nodes to execute tasks based on the data locality
- JobTracker monitors the individual TaskTrackers and the submits back the overall status of the job back to the client.
- JobTracker process is critical to the Hadoop cluster in terms of MapReduce execution

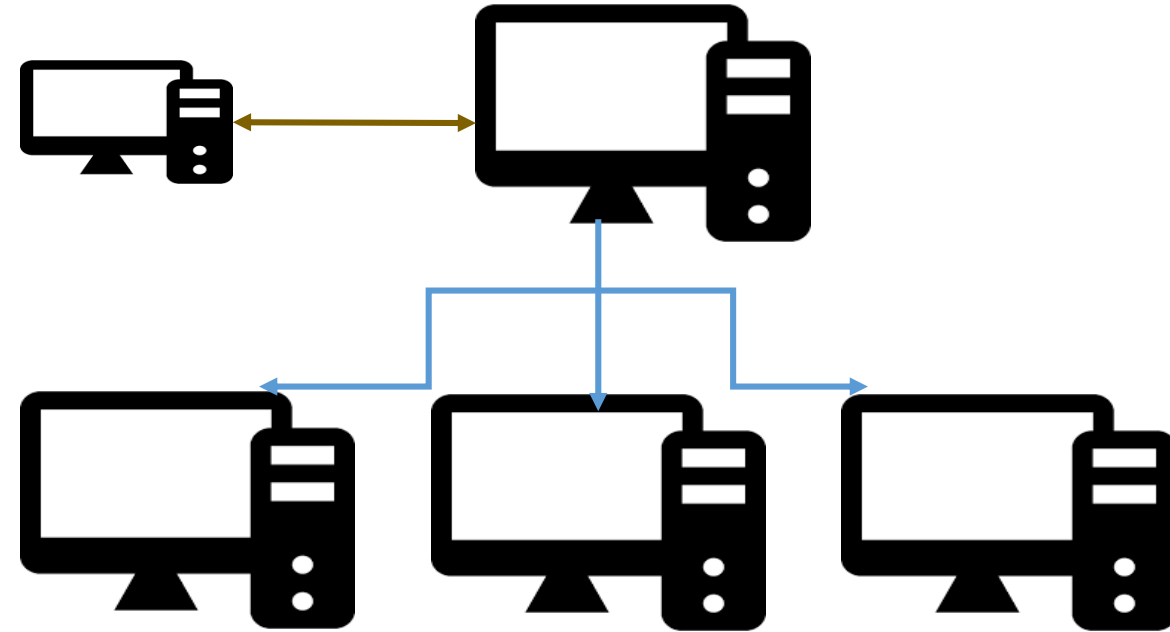
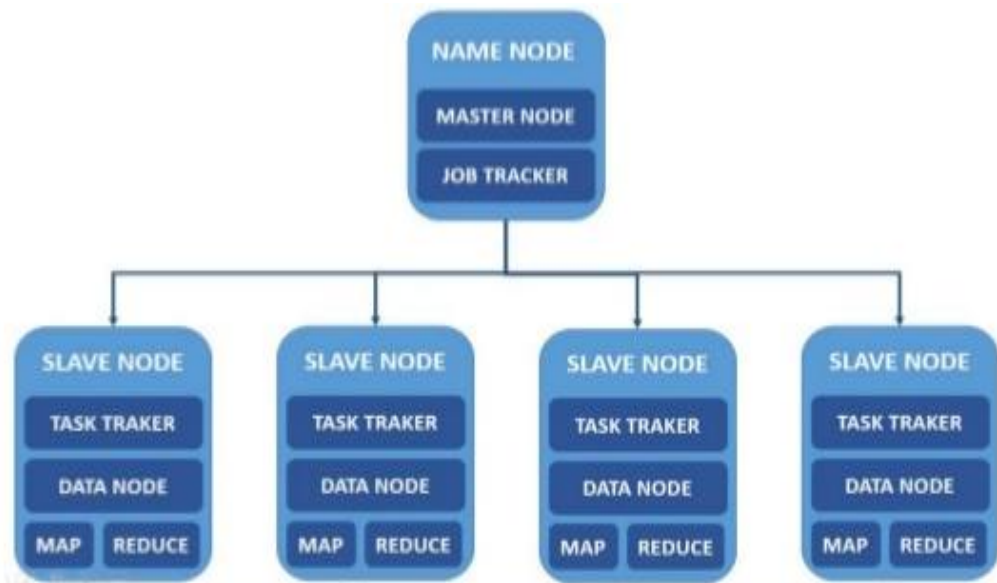
Task Tracker

- TaskTracker runs on DataNode. Mostly on all DataNodes
- TaskTracker will be in constant communication with the JobTracker signalling the progress of the task in execution.
- The JobTracker runs on MasterNode aka NameNode whereas TaskTrackers run on DataNodes

Hadoop Master / Slave Architecture



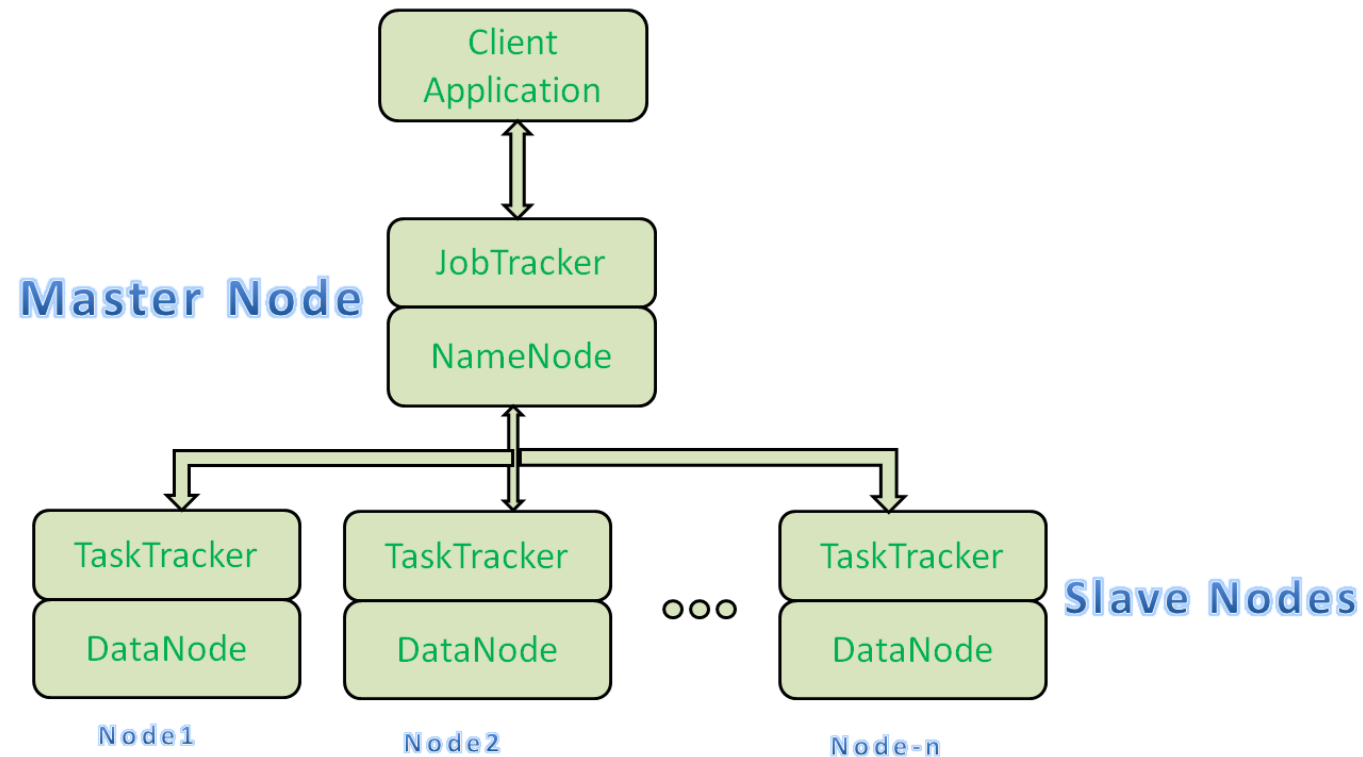
Hadoop Master / Slave Architecture



Name Node → Highly Configured Systems

Data Node → affordable low configured System

Hadoop 1.x Architecture

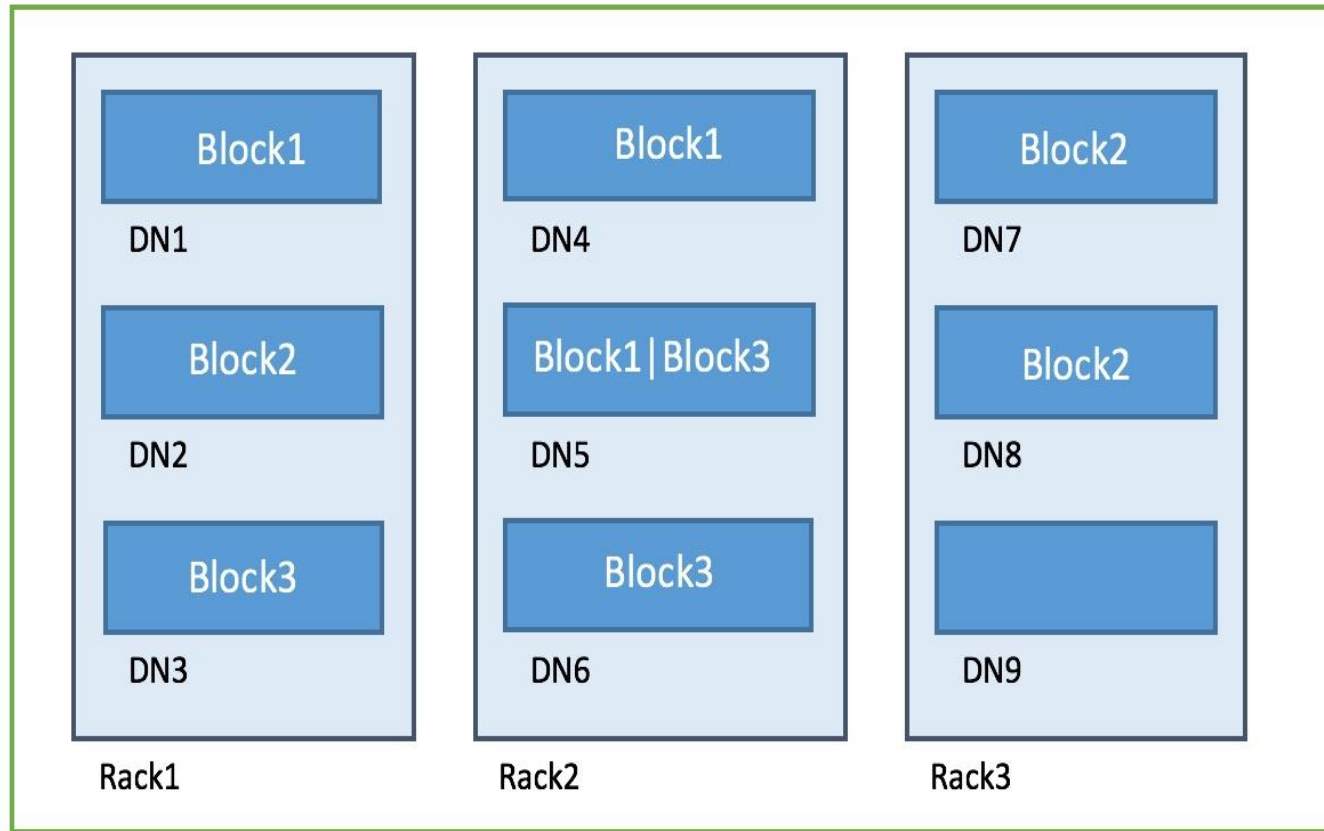


Hadoop 1.x Components Architecture

Data Racks

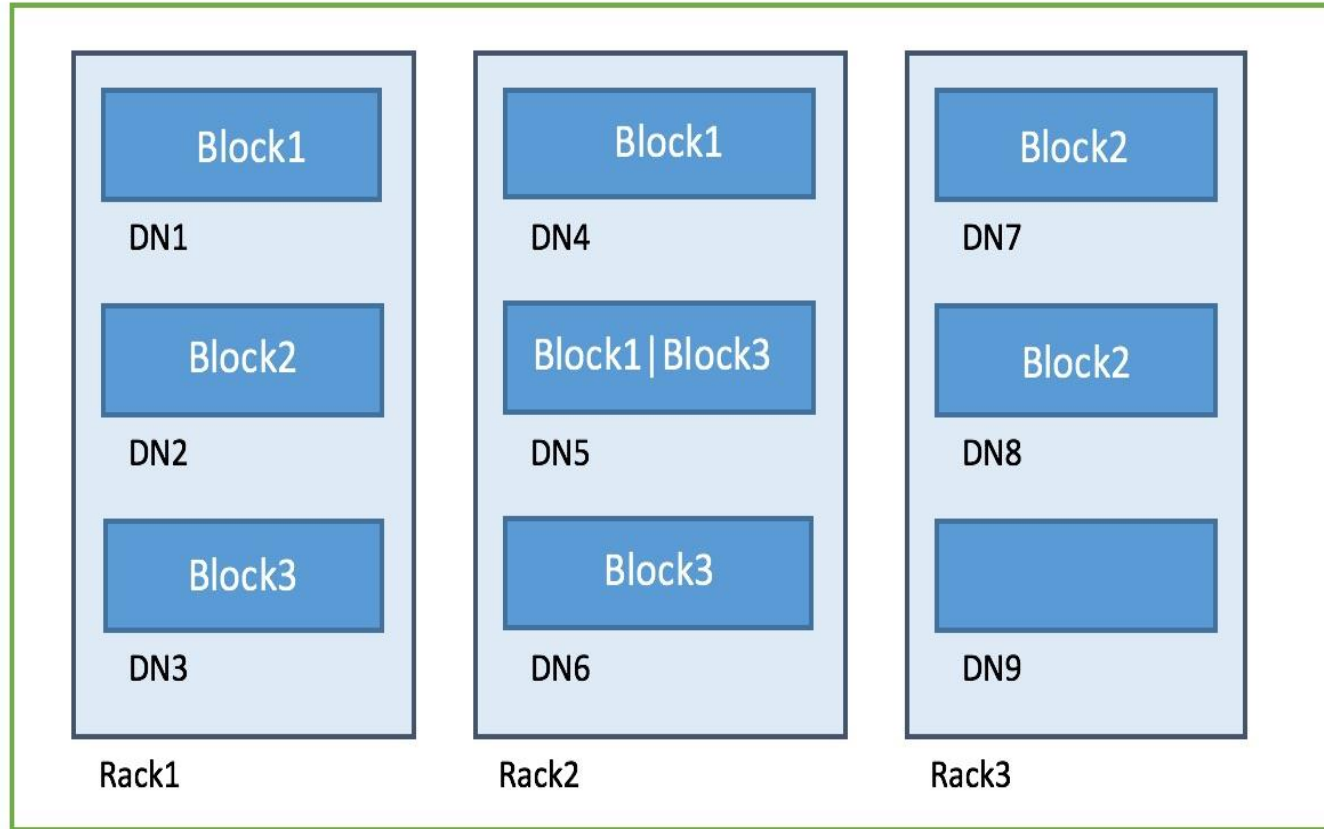


Data Storage Example



Lets say we have a file (data.txt) with 150MB

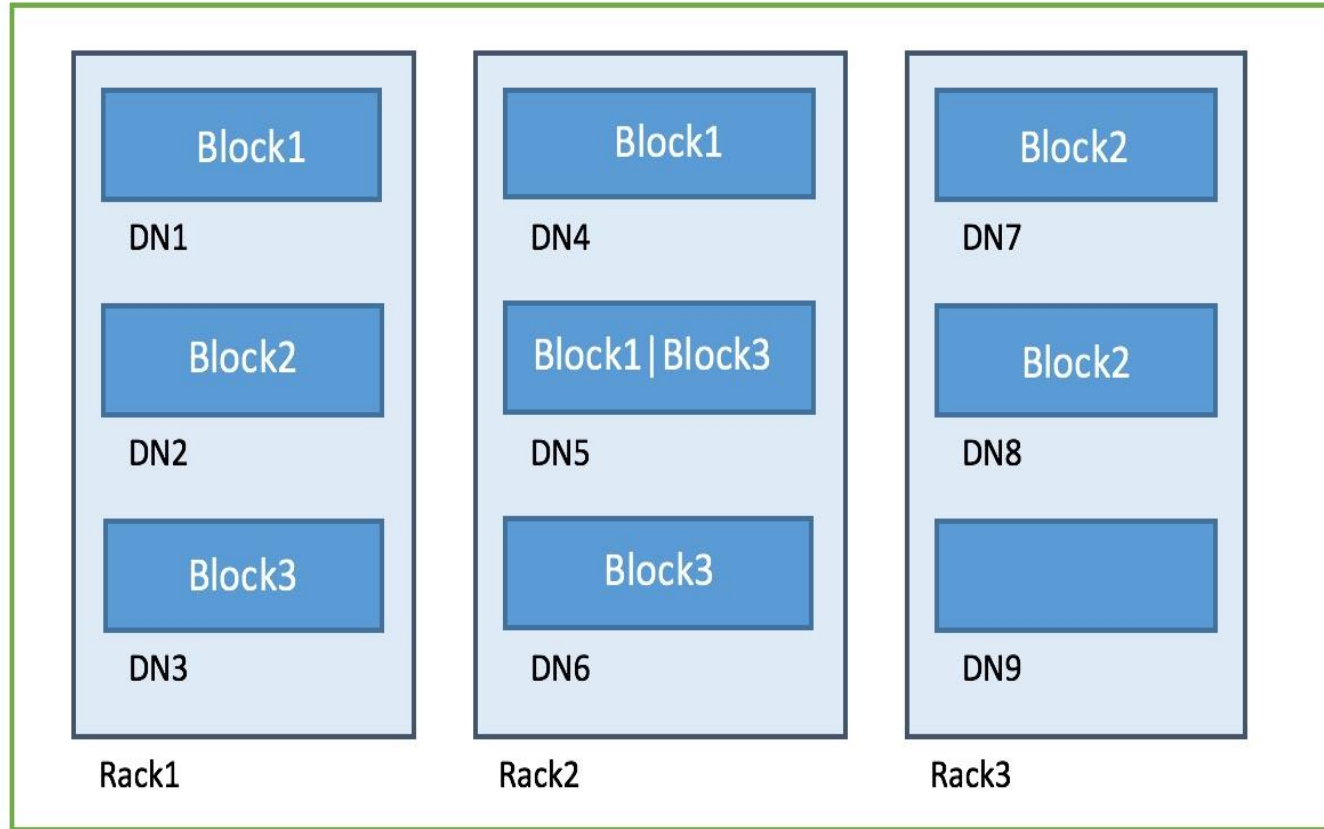
Data Storage Example



Lets say we have a file (data.txt) with 150MB

Block Size → 64MB

Data Storage Example

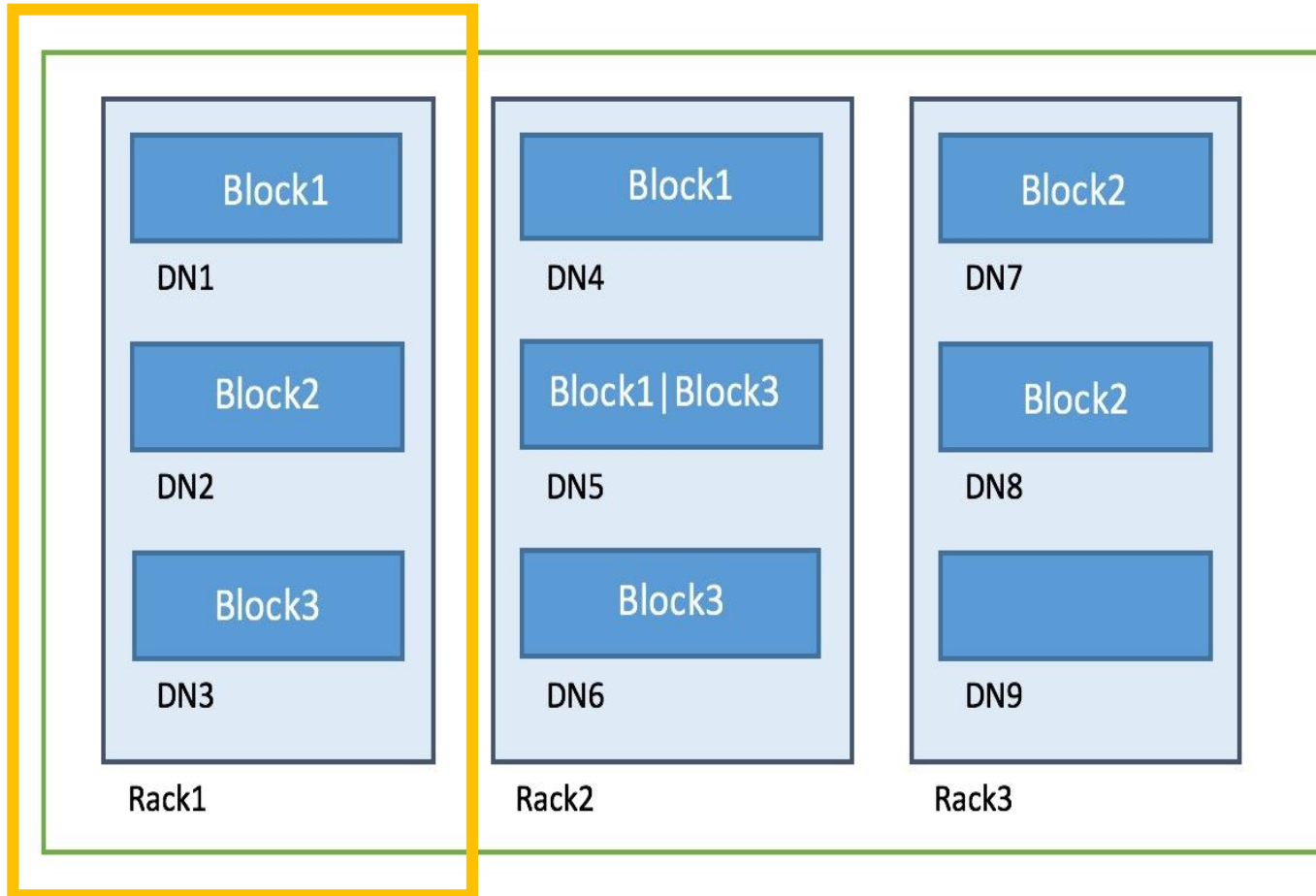


Lets say we have a file (data.txt) with 150MB

Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

Data Storage Example



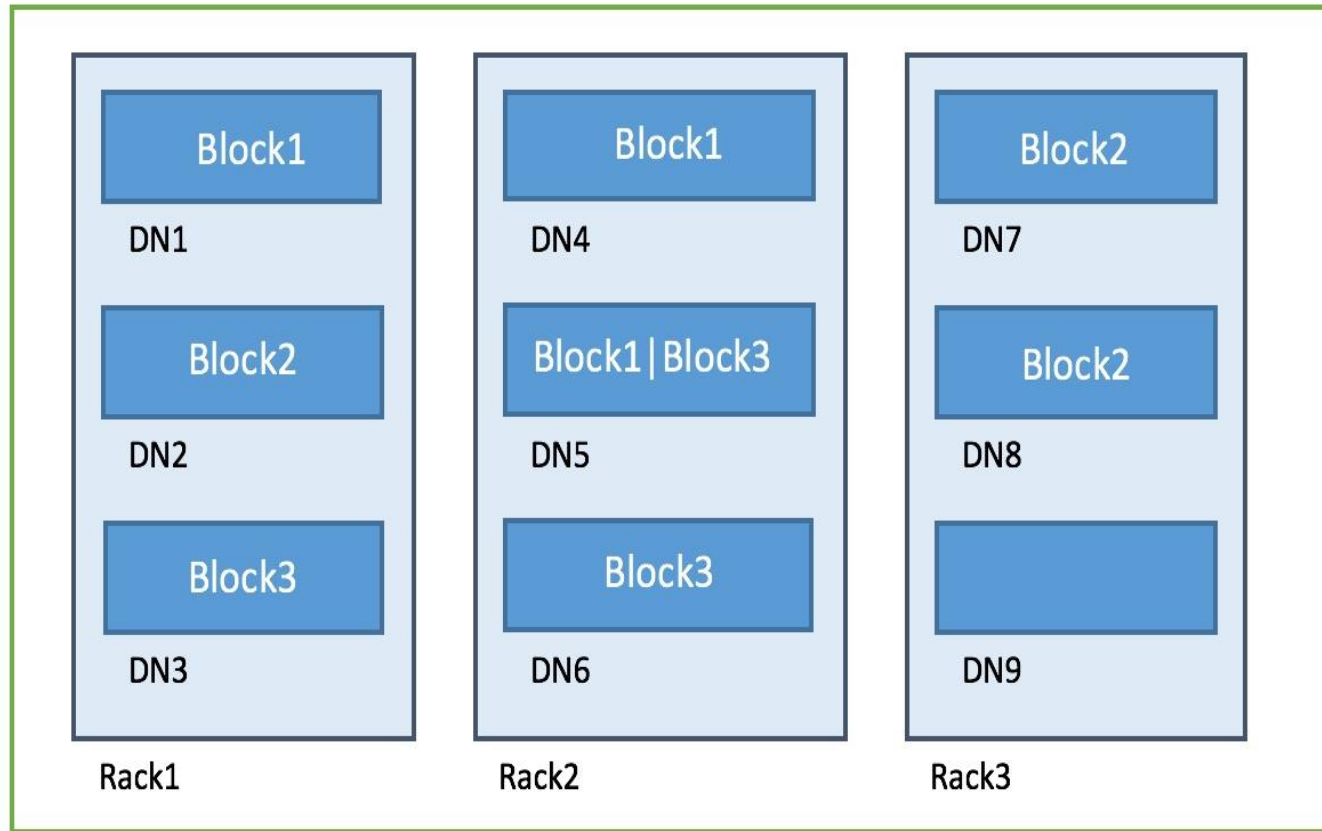
Lets say we have a file (data.txt) with 150MB

Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

Data Storage Example



Lets say we have a file (data.txt) with 150MB

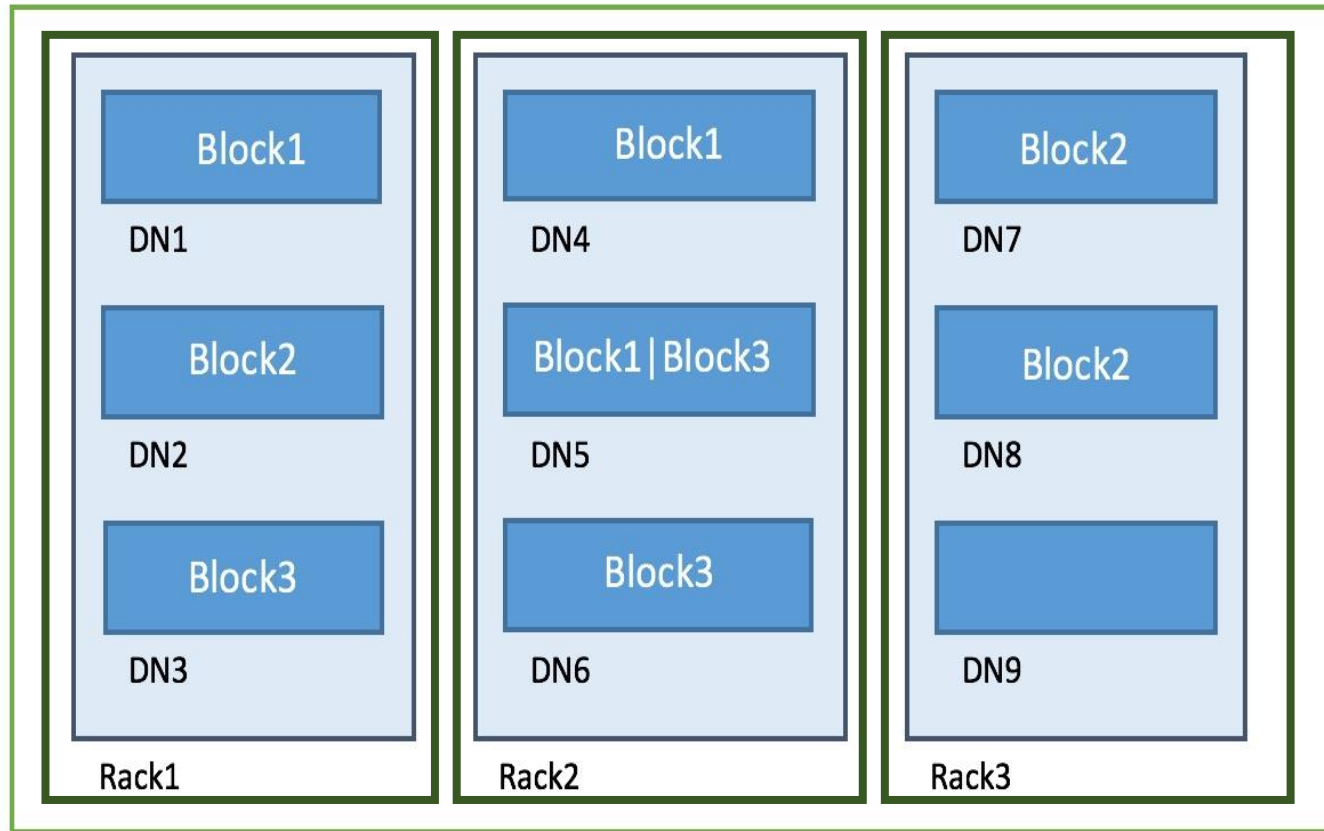
Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

→ 3 replications of 3 Blocks → 9 Data Nodes

Data Storage Example



Lets say we have a file (data.txt) with 150MB

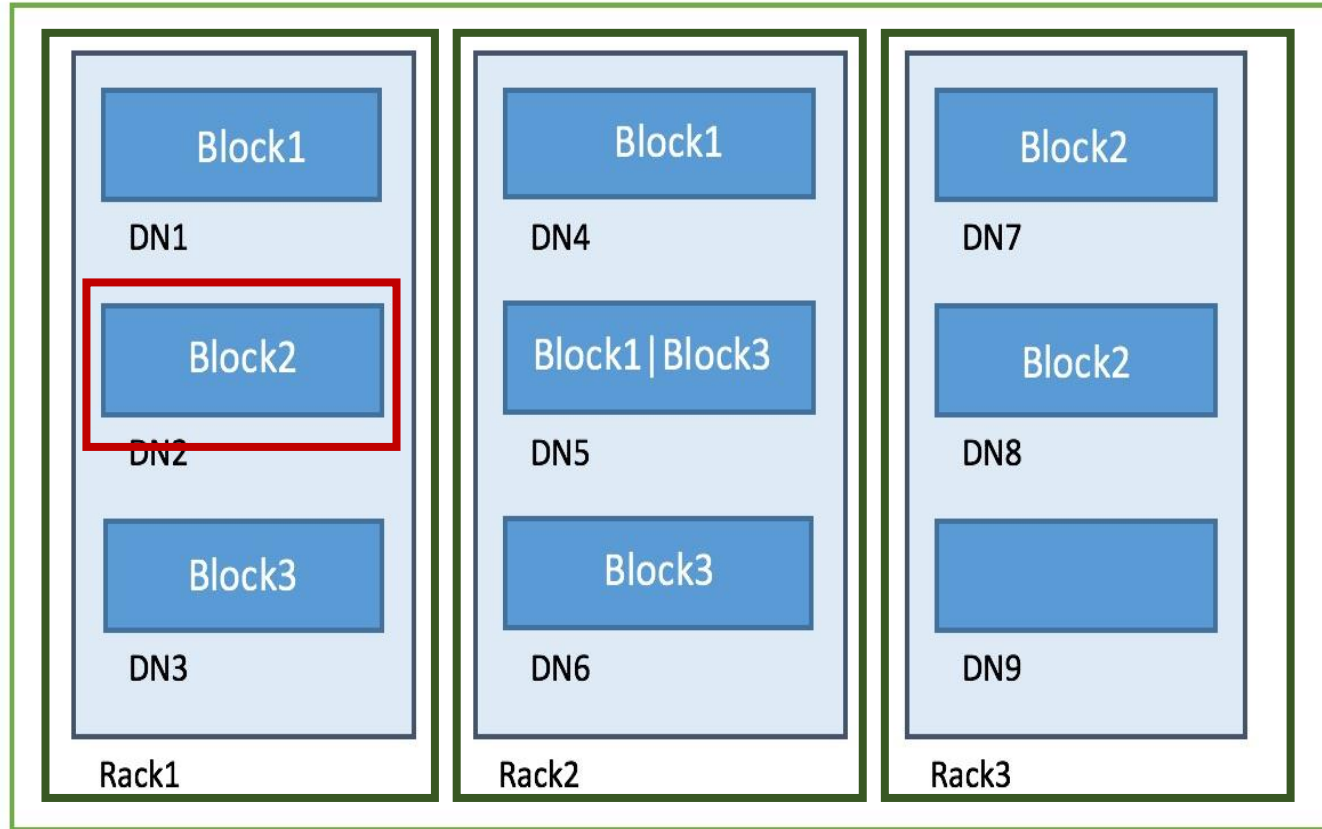
Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

→ 3 replications of 3 Blocks → 9 Data Nodes

Data Storage Example



Lets say we have a file (data.txt) with 150MB

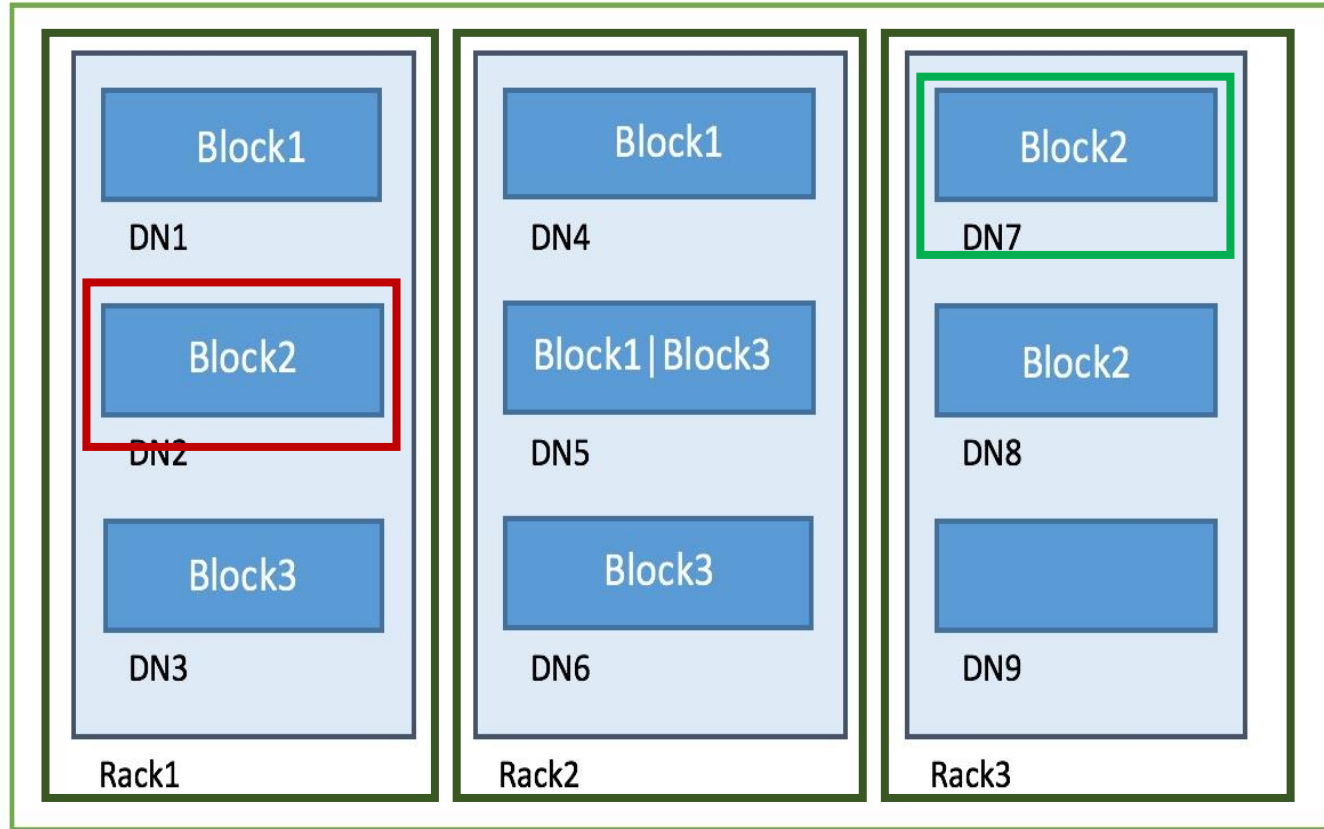
Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

→ 3 replications of 3 Blocks → 9 Data Nodes

Data Storage Example



Lets say we have a file (data.txt) with 150MB

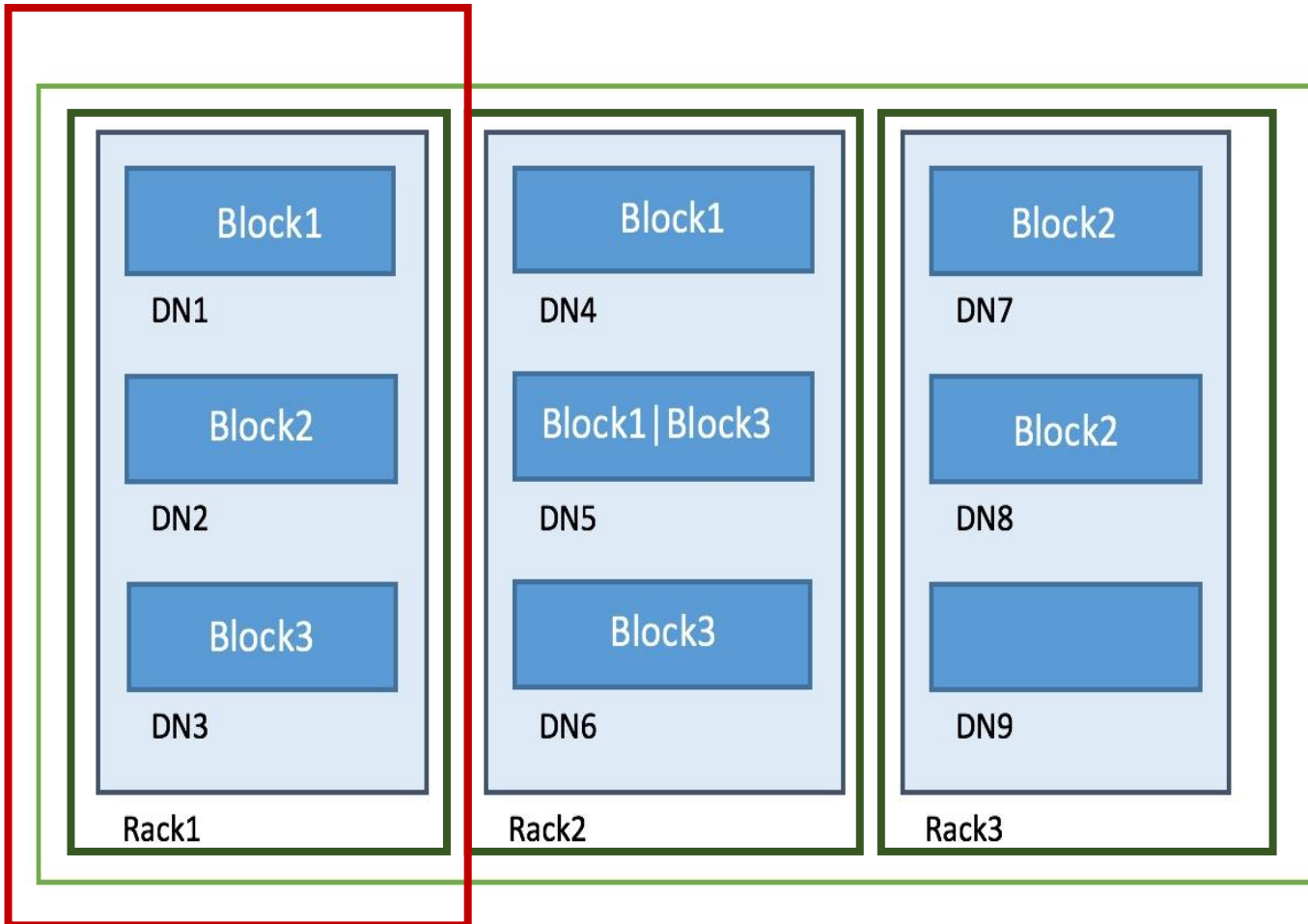
Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

→ 3 replications of 3 Blocks → 9 Data Nodes

Data Storage Example



Lets say we have a file (data.txt) with 150MB

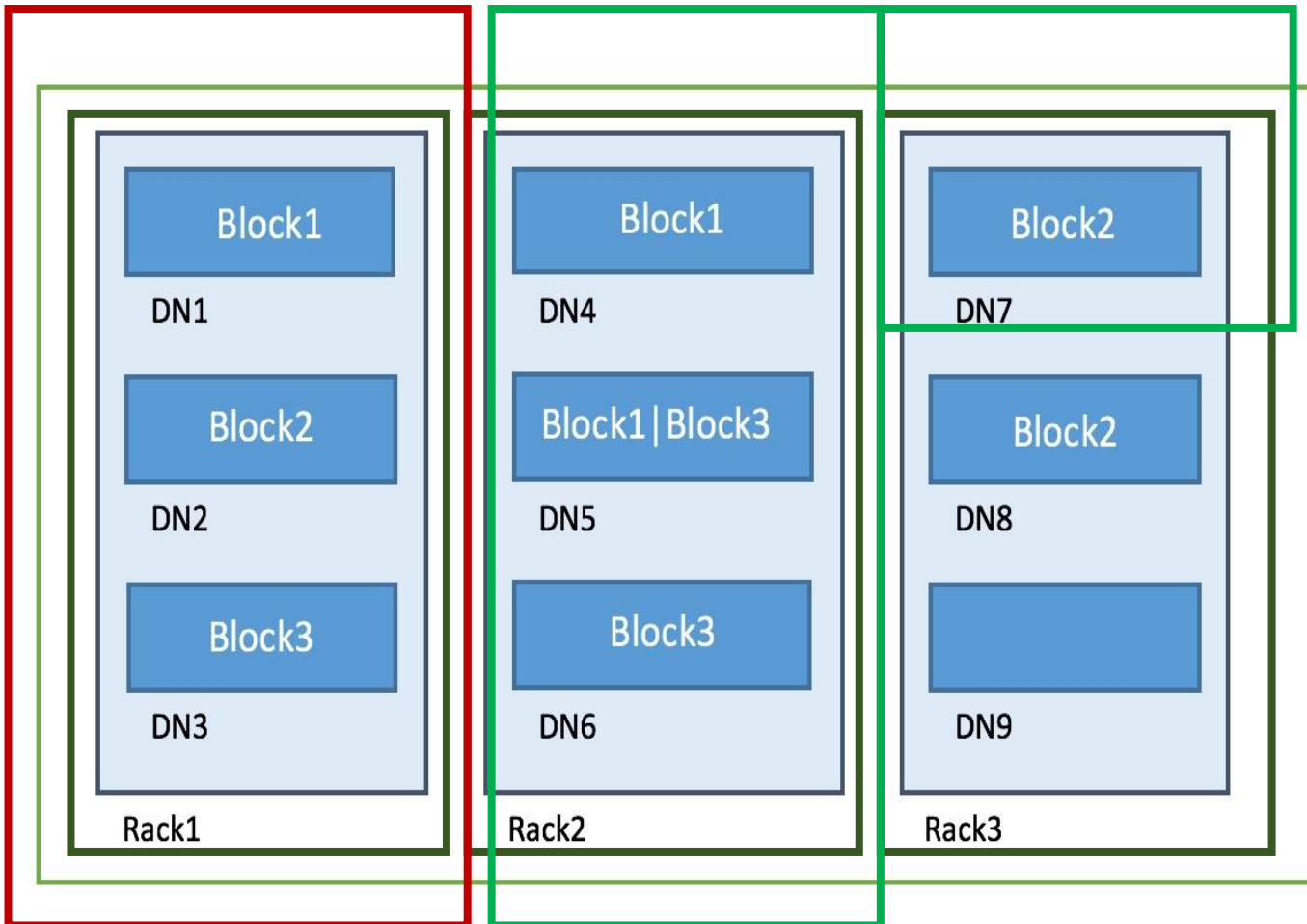
Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

→ 3 replications of 3 Blocks → 9 Data Nodes

Data Storage Example



Lets say we have a file (data.txt) with 150MB

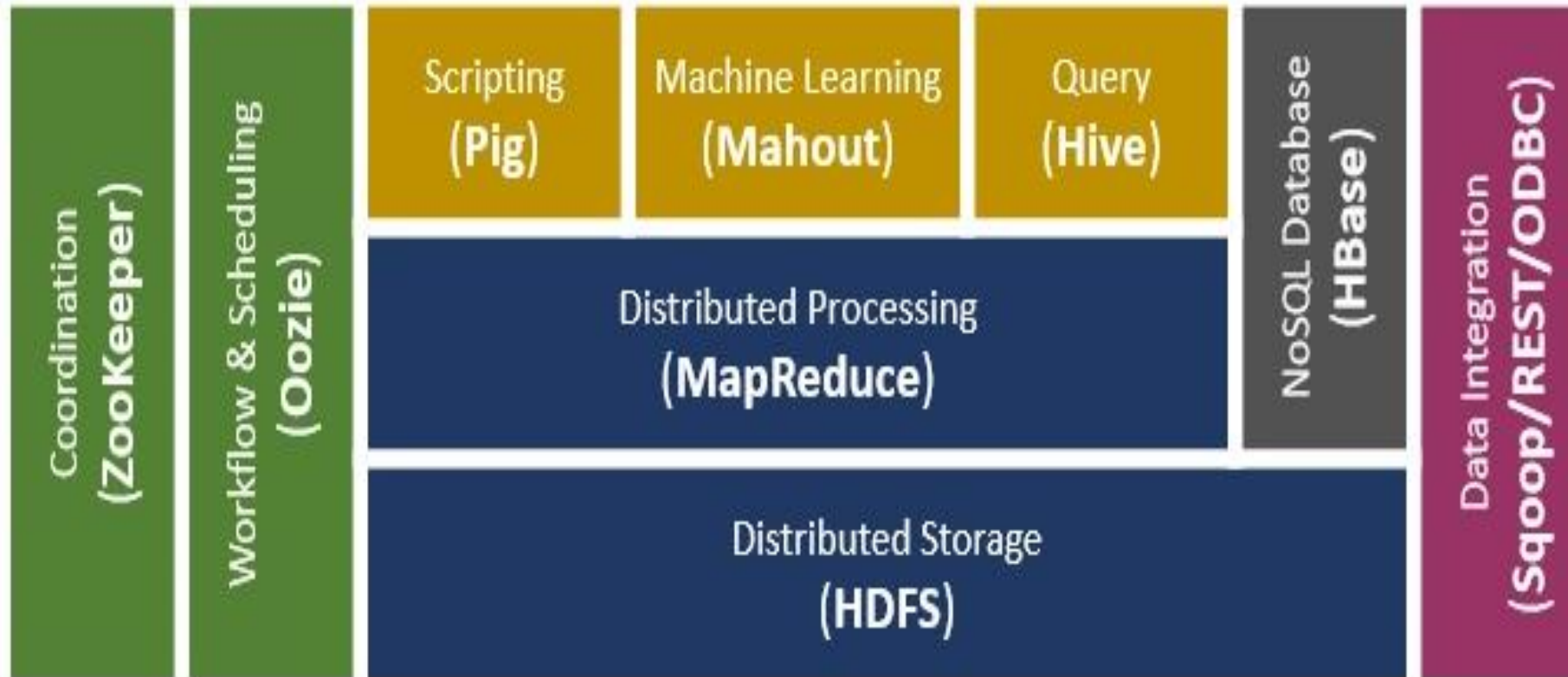
Block Size → 64MB

No of Blocks → $150\text{MB} / 64\text{ MB}$

→ 3 Blocks Is required to store for given data

→ 3 replications of 3 Blocks → 9 Data Nodes

Hadoop Eco System (Hadoop 1.x)



Limitations of Hadoop 1.x

- Only one NameNode is possible to configure i.e If NameNode fails entire cluster goes down, that is why NameNode is called as Single Point of Failure (SPOF)
- Secondary NameNode was just to take hourly backup of MetaData from NameNode
- It supports up to 4000 Nodes per Cluster.
- It has a single component : JobTracker to perform many activities like Resource Management, Job Scheduling, Job Monitoring, Re-scheduling Jobs etc.
- JobTracker is the single point of failure

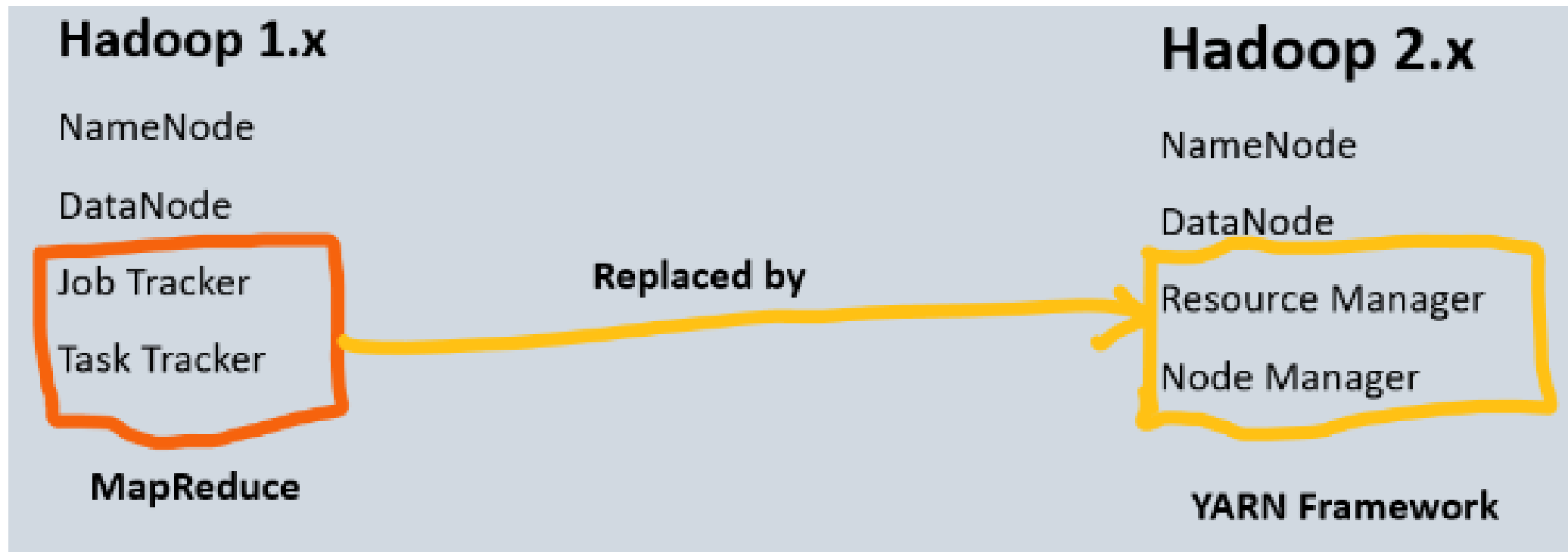
Four main improvements in Hadoop 2.0

- **HDFS Federation** – horizontal scalability of NameNode
- **NameNode High Availability** – NameNode is no longer a Single Point of Failure
- **YARN** – Yet Another Resource Negotiator
- **Resource Manager** – splits up the two major functionalities of overburdened JobTracker (resource management and job scheduling/monitoring)

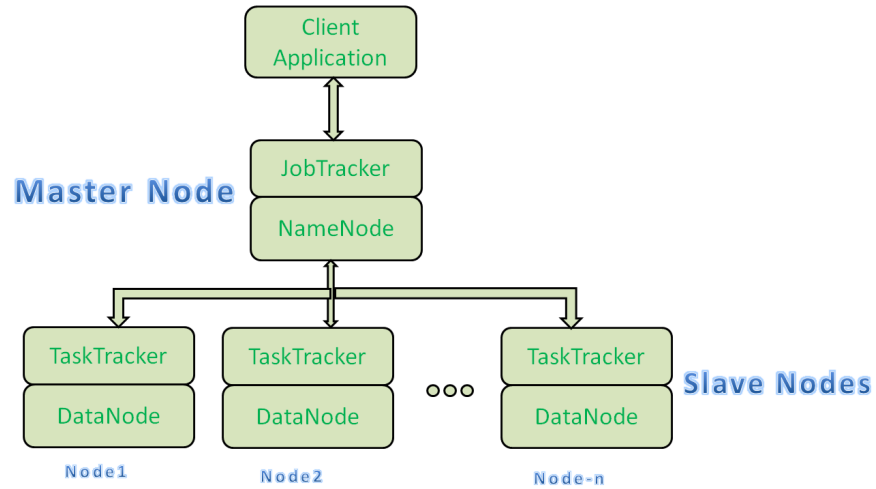
YARN

- Yet Another Resource Negotiator
- It was introduced in Hadoop 2.0 to remove the bottleneck on Job Tracker
- Main Idea to split Job Tracker responsibilities -
 - RM (Resource Manager) → (Job Scheduling)
 - AM (Application Master) → (Task Monitoring)

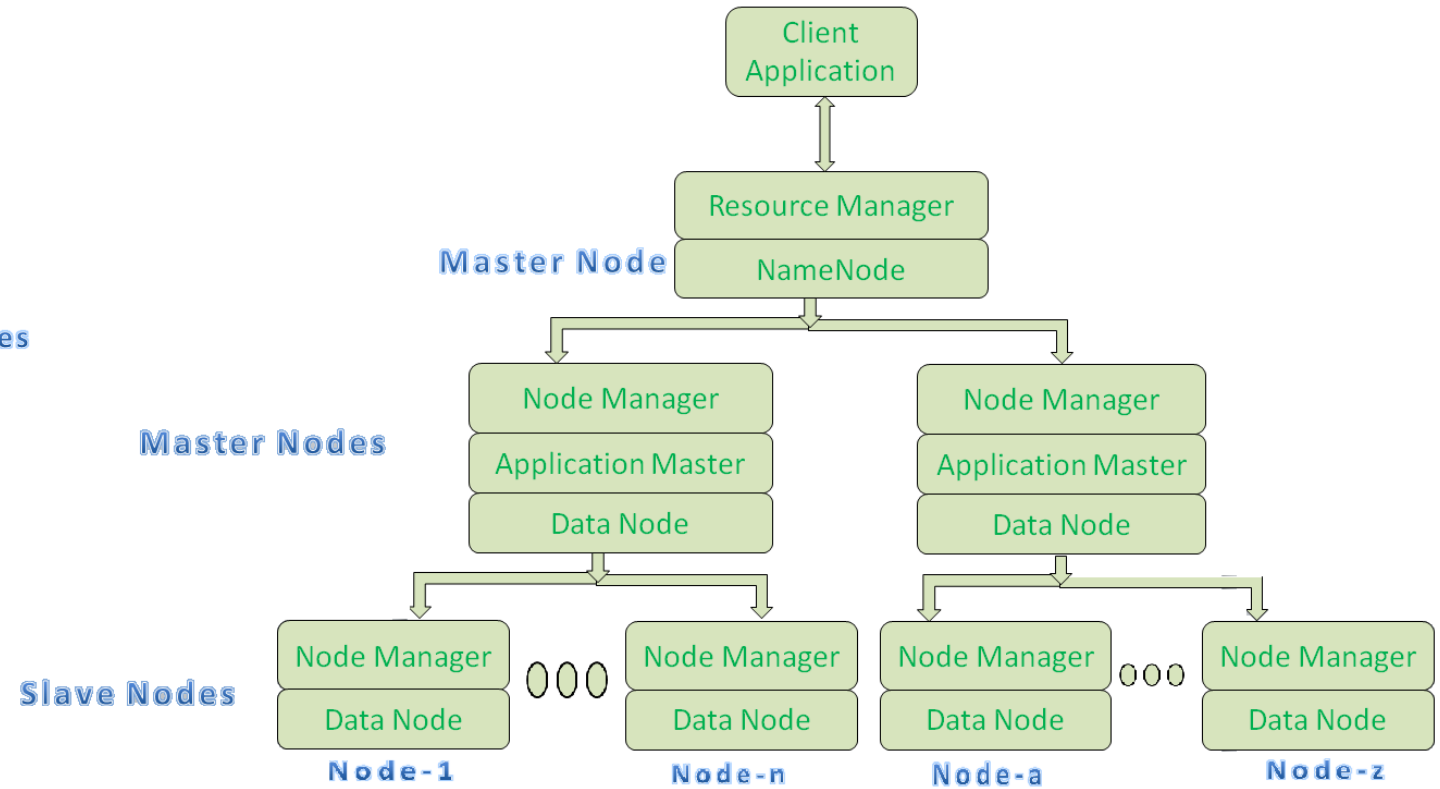
YARN



Hadoop 2.x Architecture

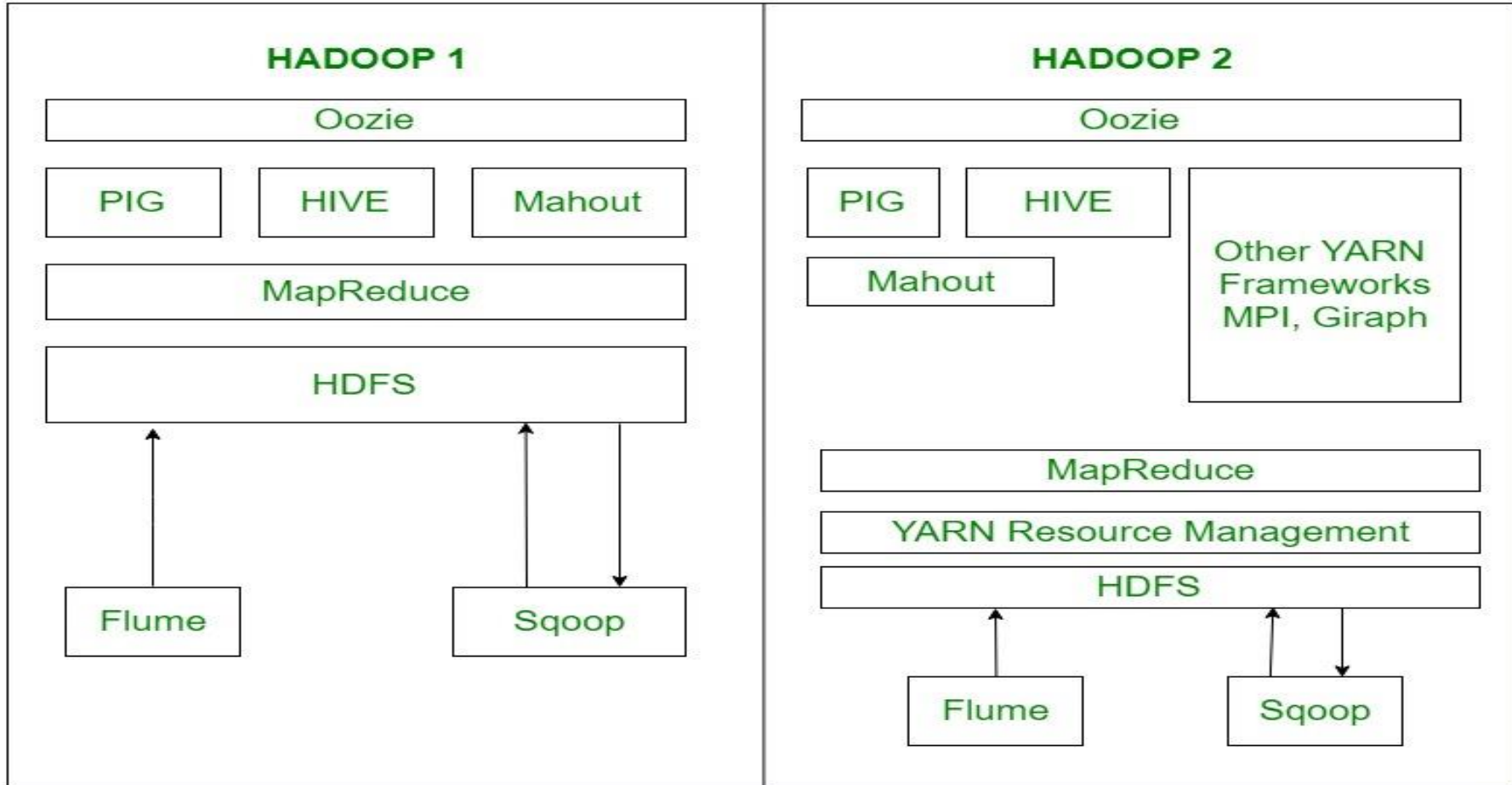


Hadoop 1.x Components Architecture

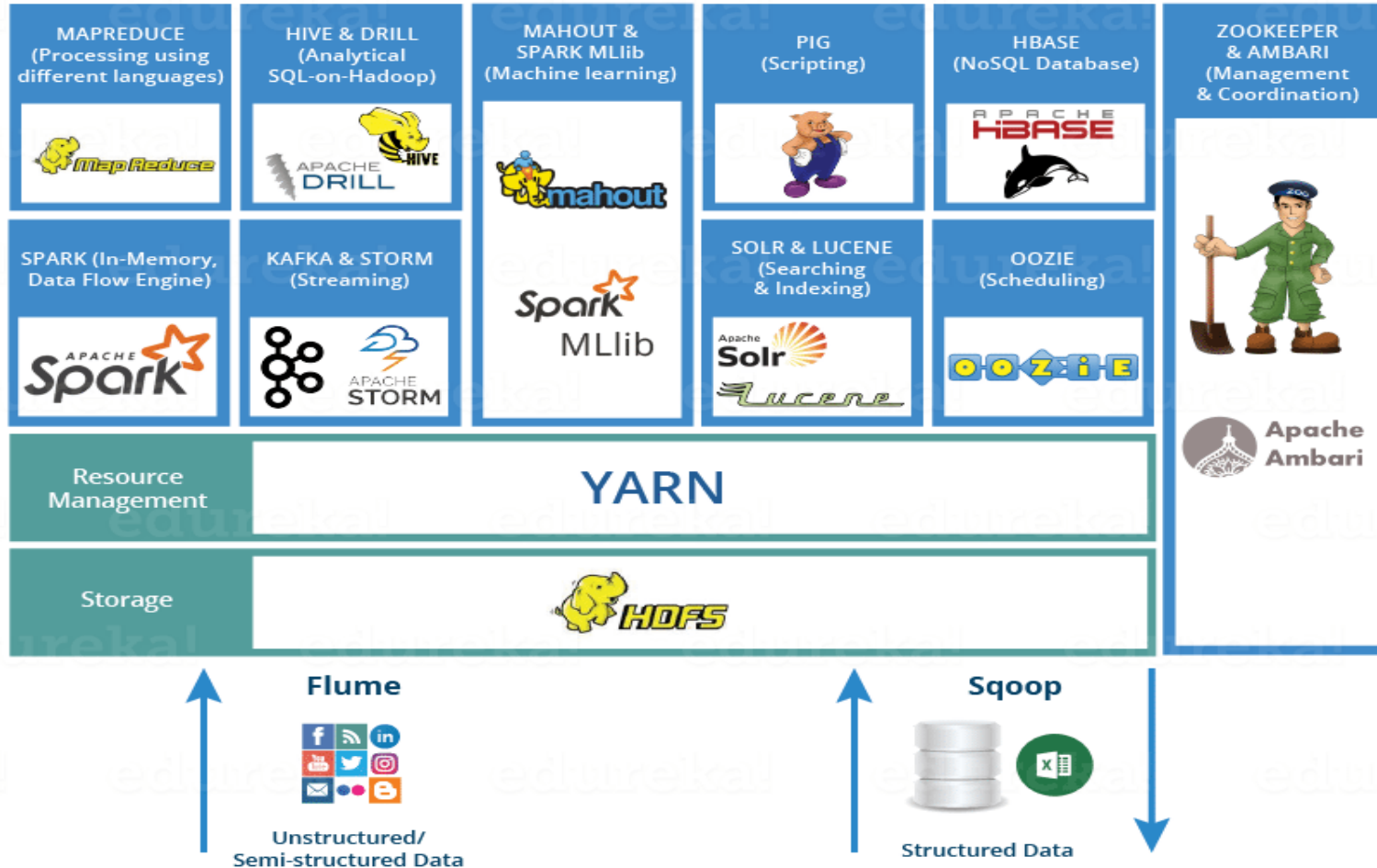


Hadoop 2.x High-Level Architecture

Hadoop 1.x and Hadoop 2.x



Hadoop 2.x Eco System



Thank You !!!

Questions??